SEO PACKAGE

1. SEO Title

SQL Constraints Explained: NOT NULL, UNIQUE, PRIMARY KEY, and FOREIGN KEY (Beginner's Guide)

2. SEO Subtitle

Learn the four SQL constraints that stop bad data before it ever reaches your database, with plain-English analogies and working examples.

3. Featured Quote

> "A constraint is a rule the database refuses to break — the last line of defense that no forgetful app code can ever sneak past."

4. Meta Description

Master the 4 essential SQL constraints — NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY — to enforce data integrity at the source. Clear examples for beginners.

5. URL Slug

`sql-constraints-not-null-unique-primary-foreign-key`

6. Keywords

SQL constraints, NOT NULL constraint, UNIQUE constraint, PRIMARY KEY, FOREIGN KEY, data integrity, referential integrity, database schema design, relational database, SQL for beginners, database validation, orphan rows, ON DELETE CASCADE, SQL tutorial, table relationships, database normalization, SQL best practices, backend engineering, data modeling, schema constraints

7. Tags

`SQL` , `Database` , `Data Integrity` , `Primary Key` , `Foreign Key` , `Schema Design` , `Backend Development` , `SQL Tutorial` , `Relational Databases` , `Software Engineering` , `LearnToCode` , `Data Modeling` , `Coding Interview`

The 4 SQL Constraints That Kill Bad Data at the Source

8. Introduction

Every backend engineer eventually learns the same hard lesson: your application code is not the only thing that talks to your database. A migration script, a colleague running a manual `UPDATE` at midnight, a forgotten cron job, a second microservice — any of them can write straight to your tables. If the only place you validate data is inside your app, then every one of those paths is a hole in the fence.

This is exactly why **SQL constraints** matter, and why they show up so often in coding interviews and system design discussions. Interviewers ask about `PRIMARY KEY` and `FOREIGN KEY` not because they want trivia, but because they reveal whether you understand *where* data integrity should live. A candidate who says "I'll validate it in the API layer" and a candidate who says "I'll enforce it in the schema and validate in the API for nicer error messages" are two very different engineers.

In this lesson we'll cover the four constraints you actually reach for every single day: **NOT NULL**, **UNIQUE**, **PRIMARY KEY**, and **FOREIGN KEY**. You'll learn what each one guarantees, how primary and foreign keys wire tables together, the classic "orphan row" trap that breaks relationships, and why enforcing rules in the database beats checking them in application code. By the end, you'll know exactly which constraint to reach for when you're designing a schema.

9. Problem Overview

Picture a table as a nightclub and each row as a person trying to get in. Without any rules, anyone walks through the door — including rows with missing names, duplicate emails, and references to users who don't exist. Chaos.

A **constraint** is a bouncer at that door. It's a rule you attach to a column (or a group of columns) that the database checks on *every* insert and update. If a row breaks the rule, the database rejects it — no

exceptions, no "we'll fix it later." In plain terms:

> A constraint is a promise your table makes about its own data, and the database enforces that promise automatically.

The four core constraints each guard a different kind of promise:

Constraint	The promise it enforces
`NOT NULL`	This column can never be empty.
`UNIQUE`	No two rows share the same value in this column.
`PRIMARY KEY`	This column uniquely and permanently identifies each row.
`FOREIGN KEY`	This value must point to a real row in another table.

The problem we're solving is **data integrity**: how do we guarantee that invalid data never gets stored in the first place, rather than discovering the mess weeks later in a broken report?

10. Example

Let's design two related tables — `users` and `orders` — and watch the constraints do their job.

```
CREATE TABLE users (
    id    INTEGER PRIMARY KEY,
    email TEXT NOT NULL UNIQUE,
    name  TEXT NOT NULL
);

CREATE TABLE orders (
    id      INTEGER PRIMARY KEY,
    user_id INTEGER NOT NULL,
    total   REAL NOT NULL,
    FOREIGN KEY (user_id) REFERENCES users(id)
);
```

Now watch what the database accepts and rejects:

```
-- ✅ Accepted: complete, unique, valid.
INSERT INTO users (id, email, name) VALUES (1, 'kai@example.com', 'Kai');

-- ❌ Rejected: name is NOT NULL, but it's missing.
INSERT INTO users (id, email, name) VALUES (2, 'sam@example.com', NULL);

-- ❌ Rejected: email must be UNIQUE, and 'kai@example.com' is taken.
INSERT INTO users (id, email, name) VALUES (3, 'kai@example.com', 'Kayla');

-- ✅ Accepted: user_id 1 points to a real user.
INSERT INTO orders (id, user_id, total) VALUES (100, 1, 49.99);
```

```
-- ❌ Rejected: user 99 does not exist — FOREIGN KEY violation.
INSERT INTO orders (id, user_id, total) VALUES (101, 99, 20.00);
```

Explaining every outcome:

- User 1 is inserted cleanly — every promise is kept.
- User 2 is rejected because `name` is declared `NOT NULL`, and we handed it a `NULL`.
- User 3 is rejected because the email already belongs to user 1, and `UNIQUE` forbids duplicates.
- Order 100 succeeds because `user_id = 1` traces back to a real user.
- Order 101 fails because there is no user 99 — the `FOREIGN KEY` refuses to let an order reference a ghost.

11. Intuition

Here's how an experienced engineer *discovers* these rules rather than memorizing them.

Start by asking a single question about each column: **"What does this data promise to always be true?"** The answer maps directly onto a constraint.

- *"Every user must have an email."* → the value must always exist → **NOT NULL**.
- *"No two users can share an email."* → the value must be one-of-a-kind → **UNIQUE**.
- *"Each user needs one permanent, unmistakable identity."* → always present **and** always unique → **PRIMARY KEY** (which is literally `NOT NULL` + `UNIQUE` fused together).
- *"An order only makes sense if it belongs to a real user."* → the value must reference something real → **FOREIGN KEY**.

That last one is the intuition worth internalizing. A foreign key is like writing a friend's phone number into your contacts — it only *means* something if that person actually exists. The `orders.user_id` column stores a pointer back to `users.id`. Follow the arrow: **orders depend on users, never the other way around.** One user can have many orders, but every order must trace back to exactly one real user. That single arrow *is* the relationship.

Once you train yourself to translate "what this data promises" into a constraint, schema design stops being guesswork.

12. "Brute Force" Solution — Validate Only in Application Code

The idea: Skip database constraints entirely and check everything in your application before writing.

```
def create_user(db, email, name):
    if not email:
        raise ValueError("Email is required")
    if not name:
        raise ValueError("Name is required")
    if db.email_exists(email):
        raise ValueError("Email already taken")
    db.insert_user(email, name)
```

Algorithm: 1. Receive the incoming data. 2. Manually run every rule in code. 3. If all checks pass, write to the database.

Advantages: - You can produce friendly, human-readable error messages ("That email is already registered"). - Rules live in one language your team already knows.

Disadvantages: - The checks only run *if the code remembers to run them*. A second script, a manual query, or another service can waltz straight past them. - Logic gets duplicated across every write path, and the copies drift out of sync. - There's a race condition: two requests can both pass the `email_exists` check before either inserts.

Time complexity: O(1) per rule, but the real cost is O(number of code paths) — you must repeat these checks everywhere, forever. **Space complexity:** O(1), but reliability approaches zero as your system grows.

13. Optimal Solution — Enforce in the Database with Constraints

The optimal approach pushes the rules down into the schema, so the database itself becomes the last line of defense. Then — optionally — you *add* app-side validation on top purely for nicer messages.

Step by step:

1. **Mark required columns `NOT NULL`.** The column can never be blank, no matter who writes to it.
2. **Mark one-of-a-kind columns `UNIQUE`.** Duplicates are rejected at the door. *(Heads up: `UNIQUE` usually still allows multiple `NULL`s — so pair it with `NOT NULL` when a value is truly required.)*
3. **Give every table a `PRIMARY KEY`.** This is the row's permanent, one-of-a-kind name tag — `UNIQUE` and `NOT NULL` combined. You get exactly one per table.
4. **Wire relationships with `FOREIGN KEY`.** A child column must point to an existing row in the parent table.

Here's the relationship in one picture:

Follow the arrow: `orders.user_id` points back to `users.id`. Delete direction matters — you can't remove a parent while children still cling to it.

Approach	Runs on every write?	Skippable?	Best-message UX
App-code checks	Only if code runs	✅ Yes	✅ Great
Database constraints	✅ Always	❌ No	⚠️ Terse

The winning move: **constraints for guarantees, app checks for friendliness.**

14. Python Code

Here's a clean, runnable example using Python's built-in `sqlite3` — no external dependencies. It builds the schema, enforces the constraints, and demonstrates the orphan-deletion trap.

```
import sqlite3


def build_schema(conn: sqlite3.Connection) -> None:
    """Create users and orders with the four core constraints."""
    # SQLite needs foreign key enforcement switched on per connection.
    conn.execute("PRAGMA foreign_keys = ON")
    conn.executescript(
        """
        CREATE TABLE users (
            id    INTEGER PRIMARY KEY,     -- UNIQUE + NOT NULL identity
            email TEXT NOT NULL UNIQUE,    -- required and one-of-a-kind
            name  TEXT NOT NULL            -- required
        );

        CREATE TABLE orders (
            id      INTEGER PRIMARY KEY,
            user_id INTEGER NOT NULL,      -- must reference a real user
            total   REAL NOT NULL,
            FOREIGN KEY (user_id) REFERENCES users(id)
        );
        """
    )
```

```
def try_insert(conn: sqlite3.Connection, sql: str, params: tuple) -> None:
    """Attempt a write and report whether the database allowed it."""
    try:
        conn.execute(sql, params)
        print(f"OK      -> {params}")
    except sqlite3.IntegrityError as err:
        print(f"BLOCKED -> {params}: {err}")


def demo() -> None:
    conn = sqlite3.connect(":memory:")
    build_schema(conn)

    try_insert(conn, "INSERT INTO users VALUES (?, ?, ?)", (1, "kai@example.com", "Kai"))
    try_insert(conn, "INSERT INTO users VALUES (?, ?, ?)", (2, "sam@example.com", None))       # NO
    try_insert(conn, "INSERT INTO users VALUES (?, ?, ?)", (3, "kai@example.com", "Kayla"))    # UN
    try_insert(conn, "INSERT INTO orders VALUES (?, ?, ?)", (100, 1, 49.99))
    try_insert(conn, "INSERT INTO orders VALUES (?, ?, ?)", (101, 99, 20.00))                  # FO

    # The orphan-row trap: user 1 still has order 100 attached.
    try_insert(conn, "DELETE FROM users WHERE id = ?", (1,))  # blocked by FK

    # Assertions turn the demo into a self-check.
    assert conn.execute("SELECT COUNT(*) FROM users").fetchone()[0] == 1
    assert conn.execute("SELECT COUNT(*) FROM orders").fetchone()[0] == 1
    print("All constraints behaved as expected.")


if __name__ == "__main__":
    demo()
```

15. Code Walkthrough

- **`PRAGMA foreign_keys = ON`** — SQLite ships with foreign key enforcement *off* by default for legacy reasons. This line is essential; without it, order 101 and the orphan delete would slip through. (PostgreSQL and MySQL/InnoDB enforce them out of the box.)
- **`id INTEGER PRIMARY KEY`** — declares the row's identity. It's automatically `UNIQUE` and `NOT NULL`.
- **`email TEXT NOT NULL UNIQUE`** — this is the "stacking" idea in action: two constraints on one column, so email is both required *and* duplicate-free.
- **`FOREIGN KEY (user_id) REFERENCES users(id)`** — ties `orders` to `users`. Any `user_id` that doesn't exist in `users.id` is rejected.
- **`try_insert`** — a thin wrapper that catches `sqlite3.IntegrityError`, the exception raised whenever a constraint is violated, so we can print `OK` or `BLOCKED` instead of crashing.

- **The `DELETE` —** attempts to remove user 1 while order 100 still references it. The foreign key protects us and refuses, preventing an orphan.
- **The `assert`s —** confirm exactly one user and one order survived, turning the example into a runnable regression check.

16. Dry Run

Let's trace the full script:

1. `build_schema` runs — two empty tables now exist with constraints armed.
2. Insert `(1, kai@example.com, Kai)` → all promises kept → **OK**. `users` has 1 row.
3. Insert `(2, sam@example.com, None)` → `name` is `NULL` but declared `NOT NULL` → **BLOCKED**.
4. Insert `(3, kai@example.com, Kayla)` → email duplicates user 1's → **BLOCKED** by `UNIQUE`.
5. Insert order `(100, 1, 49.99)` → `user_id = 1` exists → **OK**. `orders` has 1 row.
6. Insert order `(101, 99, 20.00)` → no user 99 → **BLOCKED** by `FOREIGN KEY`.
7. `DELETE FROM users WHERE id = 1` → order 100 still points at user 1 → **BLOCKED**. No orphan created.
8. Assertions confirm `users` = 1 row, `orders` = 1 row → **"All constraints behaved as expected."**

Every rejection is the database doing its job.

17. Complexity Analysis

Constraints aren't free, but they're cheap and predictable.

- `NOT NULL` — O(1) per row. It's a single presence check.
- `UNIQUE` **and** `PRIMARY KEY` — backed by an index, so each insert/update costs roughly **O(log n)** to confirm no duplicate exists, where *n* is the number of rows. The index also costs storage: **O(n)** extra space.
- `FOREIGN KEY` — each write does a lookup against the parent table's key, typically **O(log n)** thanks to the parent's index.

Why these are correct: uniqueness and referential checks rely on B-tree indexes, and a balanced B-tree lookup is logarithmic in the row count. The space overhead is the index itself — one entry per row,

hence linear. In exchange for that modest cost, you get a guarantee your application layer can never fully match.

18. Alternative Solutions

Beyond the four core constraints, two related tools deserve a mention:

- `CHECK` **constraints** — enforce a custom condition, e.g. `total REAL NOT NULL CHECK (total >= 0)` forbids negative order totals. Use this when a column has a *range* rule, not just a presence or uniqueness rule.
- `ON DELETE CASCADE` — attaches to a foreign key so that deleting a parent automatically deletes its children:

```
FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
```

Comparison: The default foreign key *blocks* a parent delete (safe, explicit — you decide what happens to children). `ON DELETE CASCADE` *cleans up* children automatically (convenient, but dangerous if you didn't mean it). Choose blocking when child data is precious; choose cascade when children are meaningless without their parent.

19. Edge Cases

- **Empty / NULL values with UNIQUE:** `UNIQUE` alone often permits several `NULL`s, since `NULL` isn't considered "equal" to another `NULL`. If a column must be both present and unique, always combine `NOT NULL UNIQUE`.
- **Adding a constraint to a table full of dirty data:** the database will *refuse* to add the constraint if existing rows already violate it. Clean the data first, then constrain.
- **Duplicates already in the column:** you cannot add `UNIQUE` until you dedupe.
- **Minimum / maximum values:** pair `CHECK` with numeric bounds to guard against negatives or overflow-prone values.
- **Deleting a parent with children:** blocked by default — delete children first, or use `ON DELETE CASCADE`.

20. Common Mistakes

1. **Assuming `UNIQUE` blocks blanks.** It usually allows multiple `NULL`s. Add `NOT NULL` when the field is truly required.

2. **Bolting constraints on after the mess.** Constraints belong in the schema *during design*, not after the table is full of invalid rows the database will reject.
3. **Forgetting to enable foreign keys in SQLite.** Without `PRAGMA foreign_keys = ON`, referential rules silently do nothing.
4. **Trying to delete a parent while children exist**, then being confused by the "wall of errors." That error is the foreign key protecting you from orphan rows.
5. **Validating only in application code.** App checks are skippable; a second script or manual query bypasses them entirely.
6. **Using more than one primary key per table.** You get exactly one. For multi-column identity, use a *composite* primary key — still one `PRIMARY KEY`, just spanning several columns.

21. Interview Questions

Likely follow-ups an interviewer will throw at you:

- What's the difference between a `PRIMARY KEY` and a `UNIQUE` constraint? *(Hint: a table has one primary key; it can have many unique columns, and unique columns may allow NULLs.)*
- What is an orphan row, and how do foreign keys prevent it?
- Explain `ON DELETE CASCADE` vs `ON DELETE RESTRICT`. When would you use each?
- Why enforce integrity in the database instead of the application layer?
- What is a composite primary key, and when is it appropriate?
- Can a foreign key reference a column that isn't unique? *(No — it must reference a primary key or unique column.)*
- What happens when you try to add a `NOT NULL` column to a table that already has rows?

22. Similar Problems

While constraints aren't a LeetCode algorithm problem, these SQL exercises drill the same relational thinking:

- **LeetCode 175 — Combine Two Tables:** practices joining a parent and child table across a foreign-key-style relationship.
- **LeetCode 181 — Employees Earning More Than Their Managers:** a self-referencing foreign key (`manager_id` → `id`), the exact "orphan" relationship in one table.
- **LeetCode 183 — Customers Who Never Order:** reasoning about which rows in one table have (or lack) references in another — the flip side of foreign keys.
- **LeetCode 196 — Delete Duplicate Emails:** the real-world cleanup you must do *before* you can add a `UNIQUE` constraint.

They're related because each one forces you to think in terms of keys, relationships, and integrity — the same muscles constraints build.

23. Key Takeaways

- A **constraint** is a rule the database enforces automatically, so bad data never makes it in.
- `NOT NULL` = no blanks. `UNIQUE` = no duplicates. `PRIMARY KEY` = the two combined into a permanent identity. `FOREIGN KEY` = one table pointing at a real row in another.
- You can **stack** constraints on a single column (e.g. `NOT NULL UNIQUE`).
- Foreign keys prevent **orphan rows**; parents can't be deleted while children reference them (unless you use `ON DELETE CASCADE`).
- **Database enforcement beats app-code validation** because constraints can never be skipped. Use app checks on top only for friendlier messages.
- Bake constraints into your schema **during design**, not after the mess.

24. Watch the Video

Reading the rules is one thing — watching them reject bad rows in real time makes them click. In the video, Kai walks through each constraint with live inserts, the orphan-delete trap, and a side-by-side comparison table. ▶️ **Watch it here: {LINK}** and follow along with your own schema.

25. About the Series

This lesson is part of **Fun with Learning Technology** — a beginner-friendly series that turns intimidating engineering topics into plain-English lessons you can actually apply. We move step by step through SQL, databases, Python, algorithms, and the day-to-day craft of software engineering, always favoring intuition over memorization. If you're building your foundations as a developer, this series is designed to grow with you, one lesson at a time.

26. Call To Action

If this made SQL constraints click, here's how to help the series grow:

- 👍 **Like** the video on YouTube so more developers find it.
- 🔔 **Subscribe** and hit the bell for the next lesson.
- 📩 **Subscribe on Substack** to get each written deep-dive in your inbox.
- 💬 **Comment** with the constraint that's tripped you up before — Kai reads and replies.

- 🔁 **Share** this with a teammate who's designing a schema right now.

SOCIAL MEDIA

LinkedIn Post

Most data-integrity bugs I've debugged had the same root cause: the only place the rules lived was inside the application code.

Here's the problem with that. Your app is never the only thing writing to your database. A migration script, a second service, a colleague running a manual UPDATE at 2am — every one of them can bypass validation that lives only in your API layer.

SQL constraints fix this by pushing the rules down into the schema itself, where nothing can skip them. There are four you'll use constantly:

• NOT NULL — the column can never be blank. • UNIQUE — no two rows share the same value. • PRIMARY KEY — NOT NULL and UNIQUE combined, giving each row a permanent identity. • FOREIGN KEY — a value must point to a real row in another table.

That last one is the one juniors underestimate. A foreign key is how `orders.user_id` guarantees every order belongs to a real user. It's also what prevents "orphan rows" — try to delete a user who still has orders, and the database stops you, because those orders would be left pointing at someone who no longer exists.

Should you still validate in application code? Yes — for friendlier error messages. But treat it as a convenience layer, not your safety net. App checks run only if your code remembers to run them. Constraints run always.

Design your integrity rules into the schema from day one. Don't bolt them on after the data is already a mess.

What's a constraint that's saved you from a nasty bug? 👇

SQL #DatabaseDesign #SoftwareEngineering #DataIntegrity #BackendDevelopment

Twitter/X Thread

1/ Your app code is NOT the last line of defense for your data.

A migration, a second service, a manual query at 2am — all of them can bypass validation that lives only in your API.

SQL constraints fix this. The 4 you'll use every day 📝

2/ First, what even is a constraint?

It's a rule you attach to a column that the database checks on every insert.

Break the rule → the row gets rejected. No exceptions.

Think of it as a bouncer that never forgets to check IDs.

3/ NOT NULL

The simplest one: this column can never be empty.

Like a form field marked with a red star — you literally can't submit it blank.

Try to insert a row with no name? Door slammed.

4/ UNIQUE

Assigned seats at a theater. Once someone's in 4B, nobody else can claim it.

No two rows share the same value in that column.

⚠️ Gotcha: UNIQUE often still allows multiple NULLs. Pair it with NOT NULL when the field is truly required.

5/ PRIMARY KEY

An employee ID badge. Everyone gets exactly one, no two people share a number.

The trick: a PRIMARY KEY is just UNIQUE + NOT NULL fused together.

One per table. It's how the database tells your rows apart.

6/ FOREIGN KEY (the important one)

Like saving a friend's number in your contacts — it only makes sense if that person exists.

`orders.user_id` must point to a real `users.id`.

Order for user 99 when there's no user 99? Rejected.

7/ Follow the arrow:

orders → users

One user can have many orders, but every order must trace back to exactly ONE real user.

That arrow IS the relationship.

8/ The classic trap: the orphan row.

Try to delete a user who still has orders and you'll hit a wall of errors.

That's the foreign key protecting you — those orders would've been left pointing at a ghost.

Fix: delete the orders first, or use ON DELETE CASCADE.

9/ "Why not just validate in my app code?"

You can — it gives nicer error messages.

But app checks only run IF your code remembers to run them. Constraints run ALWAYS.

App checks = friendly but skippable. Constraints = the guarantee nothing gets around.

10/ Recap:

• NOT NULL → no blanks • UNIQUE → no duplicates • PRIMARY KEY → the two combined = row identity • FOREIGN KEY → tables holding hands

Bake them into your schema during design, not after the mess.

Full written walkthrough + code 👇 {LINK}

Facebook Post

Ever had "bad data" sneak into your database and had no idea how it got there? 🕵️

Usually the answer is simple: the rules only lived in the app code, and something wrote around them.

SQL constraints solve this by putting the rules inside the database itself, where nothing can skip them. There are four you'll use all the time:

✅ NOT NULL — a field can't be left blank ✅ UNIQUE — no duplicate values ✅ PRIMARY KEY — a permanent, one-of-a-kind ID for each row ✅ FOREIGN KEY — links tables so an order always belongs to a real user

The foreign key is the one people underestimate — it's also what stops "orphan rows" when you try to delete something that other data still depends on.

I broke it all down with plain-English analogies and live examples. Perfect if you're just starting with databases. Watch here 👉 {LINK}

Reddit Summary

The four SQL constraints that enforce data integrity at the schema level (and why app-code validation isn't enough)

A common pattern I see in beginner backends: all data validation lives in the application layer. The issue is that your app is rarely the only writer — migrations, secondary services, and manual queries all bypass it. Constraints move the rules into the database, where they can't be skipped.

The four core ones:

- **NOT NULL** — column must always have a value.
- **UNIQUE** — no duplicate values in the column. Note: it typically still permits multiple NULLs, so combine with NOT NULL for truly required fields.
- **PRIMARY KEY** — effectively `UNIQUE + NOT NULL`, one per table, defines row identity.
- **FOREIGN KEY** — a value must reference an existing row in another table (e.g. `orders.user_id → users.id`).

Worth knowing: foreign keys prevent orphan rows. Deleting a parent that still has children is blocked by default; you either delete children first or declare `ON DELETE CASCADE`. Also, in SQLite specifically you must run `PRAGMA foreign_keys = ON` per connection or FK enforcement silently does nothing.

App-side validation still has value — mainly for friendly error messages — but it's a convenience layer, not the safety net. The database constraint is the guarantee.

Happy to answer questions on composite keys or CHECK constraints if useful.

GitHub README

````
# SQL Constraints: NOT NULL, UNIQUE, PRIMARY KEY & FOREIGN KEY

A beginner-friendly walkthrough of the four SQL constraints that enforce data
integrity **at the database level** — before bad data can ever be stored.

## Problem

Application-layer validation is skippable. Migrations, secondary services, and
manual queries can all write around it. To truly guarantee data integrity, the
rules must live in the schema, where the database enforces them on every write.

## Intuition

Ask one question of every column: *"What does this data promise to always be
true?"* The answer maps directly onto a constraint:

- "Must always exist" → **NOT NULL**
- "Must be one-of-a-kind" → **UNIQUE**
- "Needs a permanent identity" → **PRIMARY KEY** (NOT NULL + UNIQUE)
- "Must reference something real" → **FOREIGN KEY**

## Approach

| Constraint    | Guarantees                                          |
|---------------|-----------------------------------------------------|
| `NOT NULL`    | The column can never be empty.                      |
| `UNIQUE`      | No two rows share the same value.                   |
| `PRIMARY KEY` | UNIQUE + NOT NULL — a permanent row identity.       |
| `FOREIGN KEY` | The value must point to a real row in another table. |

**Relationship:** `orders.user_id` → `users.id`. Orders depend on users, never
the reverse. Deleting a parent with existing children is blocked (orphan-row
protection) unless `ON DELETE CASCADE` is set.

> ⚠️ In SQLite you must run `PRAGMA foreign_keys = ON` per connection, or foreign
> key enforcement silently does nothing.

## Python Solution

```python
import sqlite3


def build_schema(conn: sqlite3.Connection) -> None:
    conn.execute("PRAGMA foreign_keys = ON")
    conn.executescript(
        """
        CREATE TABLE users (
            id    INTEGER PRIMARY KEY,
            email TEXT NOT NULL UNIQUE,
            name  TEXT NOT NULL
        );
````
````

```
        CREATE TABLE orders (
            id      INTEGER PRIMARY KEY,
            user_id INTEGER NOT NULL,
            total   REAL NOT NULL,
            FOREIGN KEY (user_id) REFERENCES users(id)
        );
        """
    )


def try_insert(conn, sql, params):
    try:
        conn.execute(sql, params)
        print(f"OK      -> {params}")
    except sqlite3.IntegrityError as err:
        print(f"BLOCKED -> {params}: {err}")


def demo():
    conn = sqlite3.connect(":memory:")
    build_schema(conn)
    try_insert(conn, "INSERT INTO users VALUES (?, ?, ?)", (1, "kai@example.com", "Kai"))
    try_insert(conn, "INSERT INTO users VALUES (?, ?, ?)", (2, "sam@example.com", None))
    try_insert(conn, "INSERT INTO users VALUES (?, ?, ?)", (3, "kai@example.com", "Kayla"))
    try_insert(conn, "INSERT INTO orders VALUES (?, ?, ?)", (100, 1, 49.99))
    try_insert(conn, "INSERT INTO orders VALUES (?, ?, ?)", (101, 99, 20.00))
    try_insert(conn, "DELETE FROM users WHERE id = ?", (1,))
    assert conn.execute("SELECT COUNT(*) FROM users").fetchone()[0] == 1
    assert conn.execute("SELECT COUNT(*) FROM orders").fetchone()[0] == 1
    print("All constraints behaved as expected.")


if __name__ == "__main__":
    demo()
```

Run it:

```
python constraints_demo.py
```

Complexity

Constraint	Time per write	Extra space
`NOT NULL`	O(1)	O(1)
`UNIQUE` / `PRIMARY KEY`	O(log n)	O(n) index
`FOREIGN KEY`	O(log n)	O(1)*

*Relies on the parent table's existing key index.

Video

📺 Watch the full lesson: {LINK}

Article

📖 Read the complete written deep-dive: {LINK}

Part of the **Fun with Learning Technology** series. ```

Quick Review

Constraint type that forbids empty values in a column?

NOT NULL — rejects any INSERT/UPDATE leaving that column empty, guaranteeing every row has a real value there.

Difference between PRIMARY KEY and UNIQUE?

Both enforce uniqueness. PRIMARY KEY is the one row identifier (unique + NOT NULL, one per table); UNIQUE allows many per table and permits NULLs.

What does a FOREIGN KEY enforce?

A child column's value must match an existing key in the parent table (or be NULL), linking tables and blocking references to rows that don't exist.

The 'orphan' bug with foreign keys?

Deleting a parent row while children still reference it. Prevent via the FK, or use ON DELETE CASCADE/SET NULL to handle dependents.

Why constraints over app-side validation checks?

The DB enforces them on every write from any client, always. App checks are bypassable and duplicated per app; constraints are one reliable source of truth.

Can a UNIQUE column hold multiple NULLs?

In most databases yes — NULL isn't 'equal' to NULL, so several NULLs pass UNIQUE. PRIMARY KEY forbids NULL entirely.

---

sql Lesson 23: Constraints — PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL (Constraints are rules applied to columns that enforce data integrity and define the relationship between tables. By establishing these boundaries, you ensure that the data stored remains consistent, accurate, and reliable throughout the lifecycle of your application. You should implement these constraints during the design phase of your database schema to prevent invalid data states before they occur.) — free Python cheat sheet